



Java

CSC 207 Software Design

Upcoming In



Find more
on our
Quercus



Computer Science
UNIVERSITY OF TORONTO

Sep 08

DCS Award Applications Open



Apply by November 1, 2026

Sep 20

Tech@RBC Insider Series

Topic: Debug, design, deliver

Limited spots - register today!



Nov

Register for ARIA 2026



Annual showcase of applied
research and technical innovation

Upcoming In



Find more
on our
Quercus



Computer Science
UNIVERSITY OF TORONTO

DCS Career Resources

Compile Your Future



Build your skills, explore career opportunities, and connect with industry experts along the way in our workshop series!

Visit our Career Workshop & Resources page to register!

Sep 23

Resume Carousel

Real-Time Feedback from CS Alumni & Upper Years

Sep 30

Technical Interview Workshop

Strategies from an Industry Interviewer

Oct 05

Graduate School Info Session

A Positive Learning Environment

The best way to learn is to make mistakes and learn from them. To do that, everyone needs to feel safe enough to try. This means:

- Respectful communication
- Supportive and constructive feedback
- Considering the best version of an idea before critiquing it
- Active listening
- Participating without dominating
- Respecting the learning environment
- Collaborate equitably
- Have fun, while being mindful that others should be too!

L0101 and L0201 Responses to AI Prompts

Uses:

- Checking your work
- Generating code
- Writing or polishing your writing
- Explaining concepts

Negative Impacts:

- Environmental
- Learning Opportunities
- Job market

More Negative Impacts:

- Academic Integrity and potential effects on fairness
- Inequality created by inequitable access to AI

Controversies:

- Copyright
- Water and Electricity Usage

Lecture Demo

In lecture, we'll introduce various features of Java through the development of a small project in IntelliJ IDEA

These slides include additional details about the concepts being introduced.

Domain: University

Please also review the course notes sections on Java after.

Instance Variables

```
public class Circle {  
    public float radius;  
}
```

- **radius** is an **instance variable**.
- Each instance of the `Circle` class has its own `radius` variable.

Instance Variables and Static Variables

```
class Sneetch {  
    public boolean starBellied;  
    public static int howMany = 0;  
    public static final String SAYING = "Best on the Beeches.";  
}
```

- `starBellied` **is an instance variable.**
- `howMany` **is a static or class variable**
- **SAYING is static and final**

Instance Variables & Static Variables: Default Values

These variables have a default value if not initialized.

Examples:

```
double x; // 0.0
```

```
boolean b; // false
```

```
Person p; // null (like None in Python)
```

Note: local variables must be initialized before use or you get a compiler error.

toString

Method all classes inherit from class Object

Like `__str__` in Python

Override to customize the string representation of a class

```
@Override
```

```
public String toString() {...}
```

`equals`

`and`

`==`

Method all classes inherit from class `Object`

`==` for objects is like the “is” operator in Python.

Like `__eq__` in Python, but doesn't replace behaviour of `==`

Cannot change its behaviour.

Override to customize what equality means for two instances of a class

```
@Override
public boolean equals(Object o) {...}
```

`equals` for Strings

Make sure to use `.equals` when comparing Strings.

IntelliJ will warn you about this if you use `==`

See course notes for details.

Inheritance in Java

Inheritance hierarchy

- All classes form a tree called the inheritance hierarchy, with `Object` at the root.
- Class `Object` does not have a parent. All other Java classes have one parent.
- If a class has no parent declared, it is a child of class `Object`.
- A parent class can have multiple child classes.
- Class `Object` guarantees that every class inherits methods `toString`, `equals`, and others.

<https://docs.oracle.com/javase/tutorial/java/landl/subclasses.html>

Multi-part objects

Suppose class `Student` extends class `Person`.

An instance of `Student` has

- a `Student` part, with all the data members and methods of `Student`
- a `Person` part, with all the data members and methods of `Person`
- and an `Object` part

id724	Object
<i>Object instance variables and methods</i>	
Person	
<i>Person instance variables and methods</i>	
Student	
<i>Student instance variables and methods</i>	

An instance of `Student` can be used anywhere that a `Person` is legal, but not the other way around.

A `Student` is a `Person`, but a `Person` is not necessarily a `Student`

Inheritance

- Inheritance allows one class to inherit the data and methods of another class.
- In a subclass, `super` refers to the part of the object defined by the parent class.
 - Use `super. «attribute»` to refer to an attribute (data member or method) in the parent class.

Name lookup

- A subclass *can* reuse a name already used for an inherited data member or method, but it can be confusing to do so.
- Example: class `Person` could have a data member `motto` and so could class `Student`. Or they could both have a method with the signature `sing()`.
- When we construct

```
x = new Student();
```

the object has a `Student` part and a `Person` part.
- If we say `x.motto` or `x.sing()`, we need to know which one we'll get!
- In other words, we need to know how Java will look up the name `motto` or `sing` inside a `Student` object.

Name lookup rules

For a method call: `expression.method(arguments)`

- The **compiler** first checks that the method call is valid **using the declared type of expression**.
- Lookup starts in the **actual object's class**.
- If the method is not defined there, lookup continues upward through the inheritance hierarchy until it finds an implementation.
- This lookup is **dynamic**: the implementation that runs depends on the actual object's type.

Name lookup rules

For a reference to an instance variable: `expression.variable`

- The **compiler** first checks that the field access is valid **using the declared type of expression**.
- Lookup starts in the **declared type of expression**.
- If the field is not defined there, lookup continues upward through the inheritance hierarchy until it finds it.
- This lookup is **static**: the field selected depends on the reference's declared type.

Name lookup rules

Method lookup is dynamic: it uses the actual object's type.

Field lookup is static: it uses the reference's declared type.

Shadowing and Overriding

Suppose class `A` and its subclass `AChild` each have an instance variable `x` and an instance method `m`.

- Method `m` in `A` is **overridden** by method `m` in `AChild`.
 - Dynamic lookup makes overriding useful for specializing behaviour.
- Instance variable `x` in `A` is **shadowed** by instance variable `x` in `AChild`.
 - Static lookup makes shadowing confusing and rarely useful.
 - Avoid instance variables with the same name in a parent and child class.

id725	Object
<i>Object instance variables and methods</i>	
A	
int x void m()	
AChild	
int x void m()	

Casting for the compiler

- The Java compiler uses the type of the reference to determine whether a statement is valid.
- This code doesn't compile:

```
Object o = new String("hello");  
char c = o.charAt(1);
```
- The compiler does not keep track of object types, only variable types.
Remember: the compiler doesn't run the code – it can only look at the type of `o` to determine whether it is legal.
- To make the code compile, we need to **cast** `o` as a `String`:

```
char c = ((String) o).charAt(1);
```
- This is dangerous. Why?

Checking the object type

At runtime, we can use operator `instanceof` to determine whether an object really is an instance of the specified type.

```
Object o = "Yo";  
if (o instanceof String) {  
    char c = ((String) o).charAt(1);  
}
```

Constructors

- A constructor has:
 - the same name as the class
 - no return type (not even `void`)
- A class can have multiple constructors, but their signatures must be different.
- If you define no constructors, the compiler supplies one with no parameters and no body.
 - But if you define any constructor for a class, the compiler will no longer provide the default no-arg constructor.

this

- `this` is a reference to the current object
- It's like `self` in Python.
- It can be useful to disambiguate between an instance variable and a parameter with the same name.

<https://docs.oracle.com/javase/tutorial/java/javaOO/thiskey.html>

Class design tips

- Everything should be as private as possible.
 - Always make instance variables private. Always!
- Create public methods only when necessary. These provide an API.
 - For each instance variable, decide whether to provide a public getter, a setter, or both. (You can ask IntelliJ to write these for you!)
- Look at how the class will be used. Are there additional methods that would make that easier?
- If you create any helper methods, make them private.

Encapsulation

- Think of your class as providing a service.
 - We provide access to information through a well-defined interface: the public methods of the class.
 - We hide the implementation details.
- What is the advantage of this “encapsulation”?
 - We can change the implementation — to improve speed, reliability, or readability — and *no other code must change!*

Access Modifiers

- Classes can be declared public or package-private.
- Members of classes can be declared public, protected, package-private, or private.

Modifier	Class	Package	Subclass	World
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes	No
default (package private)	Yes	Yes	No	No
private	Yes	No	No	No

Constructors and inheritance

- If the first step of a constructor is `super («arguments»)`, the appropriate constructor in the parent class is called.
 - Otherwise, the no-argument constructor in the parent is called.
- Net effect on order if, say, A is parent of B is parent of C?
- Which constructor should do what? Good practice:
 - Initialize your own variables.
 - Count on ancestors to take care of theirs.

What happens when we create an object?

1. The new keyword is used to create an object.
2. Memory is allocated and instance variables are initialized to their default values (in all parts of the object).
3. The constructor begins execution.
 - Its first statement must be either `this(...)` or `super(...)`.
 - If neither is written, Java inserts `super()`.
 - Superclass initialization occurs before subclass initialization.
 - Instance variable initializations are executed in the order they appear.
 - The rest of the constructor body executes.

Java Object Creation Trace

```
Student s = new Student("Alice");
```

Step 1:

Step 2:

...

```
public class Person {  
    private String name = "Unknown";  
    public Person(String name) {  
        this.name = name;  
    }  
}
```

```
public class Student extends Person {  
    private int studentNumber = -1;  
    public Student(String name) {  
        this(name, 0);  
    }  
}
```

```
    public Student(String name, int studentNumber)  
    {  
        super(name);  
        this.studentNumber = studentNumber;  
    }  
}
```

Java Object Creation Trace

```
Student s = new Student("Alice");
```

1. Allocate memory for the entire Student object
2. Default values: name=null, studentNumber=0
3. Enter Student(String name)
4. Call this(name, 0)
5. Enter Student(String name, int studentNumber)
6. Call super(name)
7. Person field init: name="Unknown"
8. Person constructor: name="Alice"
9. Student field init: studentNumber=-1
10. Student constructor: studentNumber=0

```
public class Person {  
    private String name = "Unknown";  
    public Person(String name) {  
        this.name = name;  
    }  
}
```

```
public class Student extends Person {  
    private int studentNumber = -1;  
    public Student(String name) {  
        this(name, 0);  
    }  
}
```

```
    public Student(String name, int studentNumber)  
    {  
        super(name);  
        this.studentNumber = studentNumber;  
    }  
}
```

Interfaces & Abstract Classes

Abstract classes and interfaces

- A class may define methods without giving a body. In that case:
 - Each of those methods must be declared `abstract`.
 - The class must be declared `abstract` too.
 - The class can't be instantiated.
- A child class may implement some or all the inherited abstract methods.
 - If not all, it must be declared `abstract`.
 - If all, it's not abstract and so can be instantiated.
- If a class is completely abstract, we may choose instead to declare it to be an `interface`.

Interfaces

An Interface is like an abstract class, but:

- It is separate from the inheritance hierarchy, so a class can **implement** any number of interfaces.
 - E.g., `class A extends B implements InterfaceC, InterfaceD`
- It can't have instance variables (so no state information).
- Its methods must be either abstract or provide a default implementation. Syntactically, we don't have to use the `abstract` keyword since methods are assumed to be abstract — the `default` keyword is used if the interface does provide a default implementation.

Example: `java.util.List` — "To be a List, here are the methods you must support."

An interface may also extend another interface.

The Programming Interface

- The "user" for almost all code is a programmer. That user wants to know:
 - ... what kinds of object your class represents
 - ... what actions it can take (methods)
 - ... what properties your object has (getter methods)
 - ... what guarantees your methods and objects require and offer
 - ... how they fail and react to failure
 - ... what is returned and what errors are raised

Interface Left, Class Right

```
List<Integer> ls = new List<>();           // Fails
```

```
List<Integer> ls = new ArrayList<>();
```

We can choose a different implementation of `List` at any point.

The compiler guarantees that we don't call any methods that are in `ArrayList` but not in `List`.

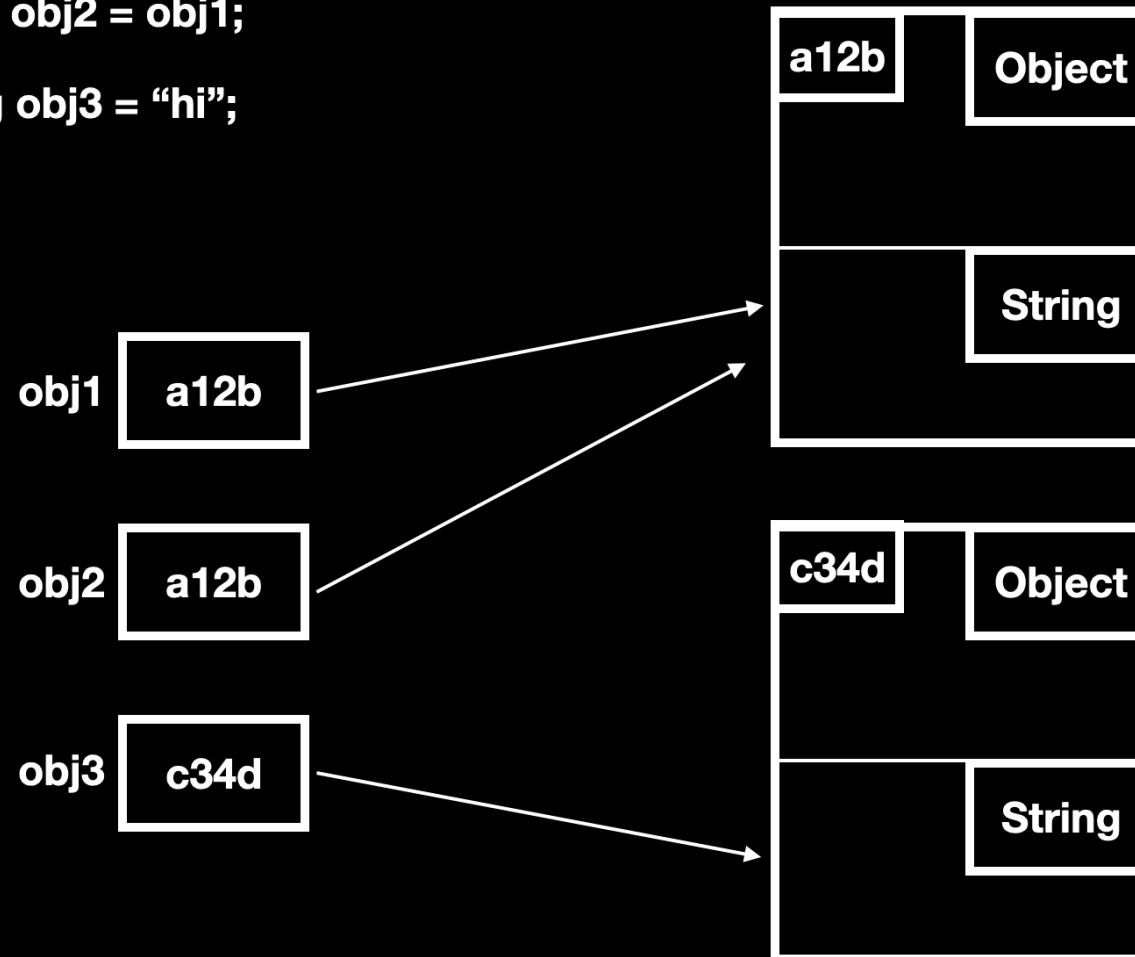
Java Memory Model: Primitive & Reference Types

Reference Types

String obj1 = "hello";

String obj2 = obj1;

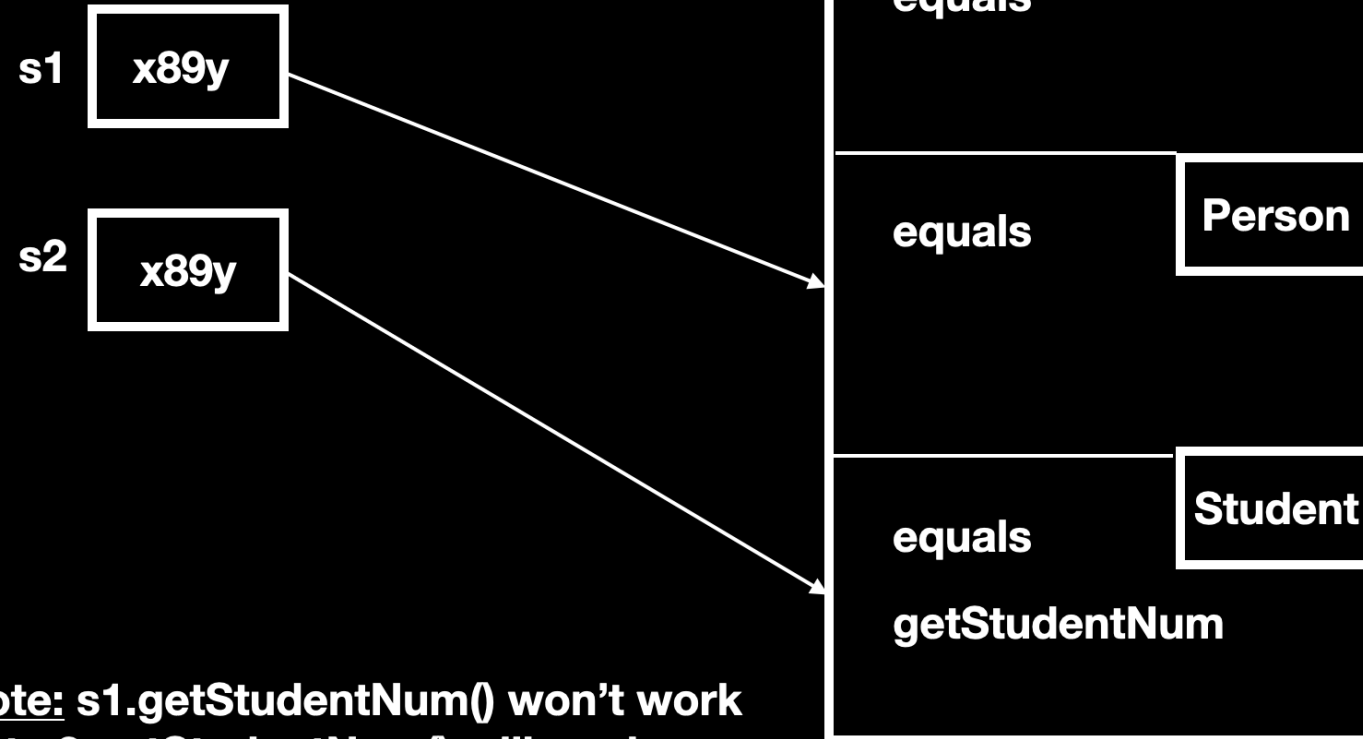
String obj3 = "hi";



Casting

```
Person s1 = new Student("A B");
```

```
Student s2 = (Student) s1;
```



Note: `s1.getStudentNum()` won't work
but `s2.getStudentNum()` will work.

Primitive vs Reference Types

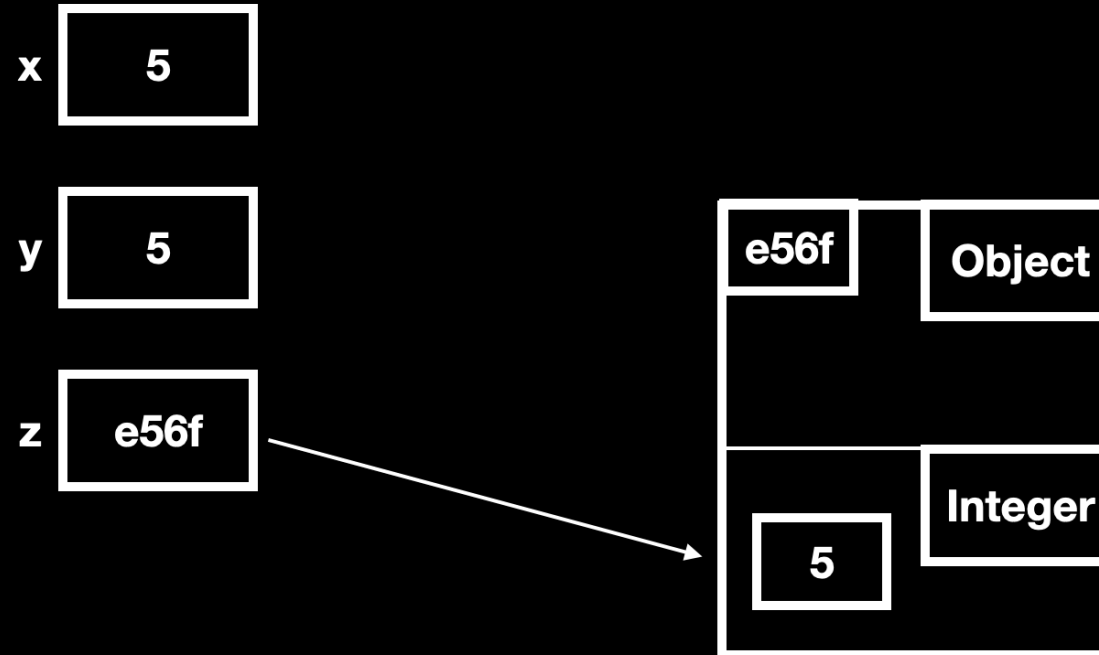
- In the previous slide, `s1` contains the address where the instance of a class is actually stored.
- The fact that `s1` points at its "value" rather than directly storing the value means that `s1` has a reference type.
- On the next slide, we will see that `x` and `y` store their actual values, not an address that points to its value. This makes `int` a primitive type.
- You can read more about primitive types in Java here:
<https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html>

Primitive Types

```
int x = 5;
```

```
int y = x;
```

```
Integer z = new Integer(5);
```



Wrapper Classes

- Each primitive type has a wrapper class.
 - For example, integers can be stored as a primitive value (`int a = 2;`) or as an Integer object (`Integer b = new Integer(2);`)
- Why do we need both?
 - Primitive values use less memory, but do not have constructors or methods
 - If you want to store a primitive value in a *generic* collection such as an `ArrayList`, we need a reference type to wrap the object in.
- Can we go between the two?
 - Yes! We can do this intentionally. Java does it automatically. For example:
`int x = 2;`
`Integer y = new Integer(x);`

Autoboxing and unboxing

Sometimes Java can guess what you intended to do and will autobox (automatically make an object to store the primitive) or unbox (treat the value inside an object like its corresponding primitive)

```
int x = 2;  
int y = 2;  
Integer z = new Integer(2);  
Integer w = z;
```

`x == y` will evaluate to `true` because `==` compares whatever is stored with the variable name. Since `x` and `y` are primitives, the value 2 is stored.

`z.equals(w)` will evaluate to `true` because the `Integer` `equals` method compares values.

`z == w` will evaluate to `true` because they both contain the same address of the `Integer` object that was created on the third line of code above.

`x == z` will evaluate to `true` because Java will automatically unbox `z` and compare its value instead of its address.

`x.equals(z)` will not run because Java will not know to autobox `x` into an `Integer` object. The parameter type is `Object`.

Documentation & Style Conventions

Javadoc

- Like a Python docstring but placed above the method.

```
/**  
 * Replace a square wheel of diagonal diag with a round wheel of  
 * diameter diam. If either dimension is negative, use a wooden tire.  
 * @param diag Size of the square wheel.  
 * @param diam Size of the round wheel.  
 * @throws PiException If pi is not 22/7 today.  
 */  
public void squareToRound(double diag, double diam) { ... }
```

- Javadoc is written for classes, member variables, and member methods.
- This is where the Java API documentation comes from!
- <https://www.oracle.com/ca-en/technical-resources/articles/java/javadoc-tool.html> provides an in-depth discussion of how to write documentation for Java code.

Java naming conventions

- The Java Language Specification recommends these conventions
- Generally: Use `camelCase` not `pothole_case`.
- Class name: A noun phrase starting with a capital.
- Method name: A verb phrase starting with lower case.
- Instance variable: A noun phrase starting with lower case.
- Local variable or parameter: ditto, but acronyms and abbreviations are more okay.
- Constant: all uppercase, `pothole`.
 - E.g., `MAX_ENROLMENT`

CheckStyle

- CheckStyle is like PythonTA from first year
- Automated checks to ensure style and coding conventions are being followed
- Can be configured to reflect the best practices of your team
 - Teams often set up pre-commit hooks in their version control systems to enforce that style checks pass before code can be committed.
- There is a CheckStyle plugin for IntelliJ IDEA; Maven can also be configured to run CheckStyle on your project code.

Homework

Continue working through the course note readings on Java and complete their associated exercises [for credit].

Take the Quercus quizzes to additionally check your understanding.

Start working through the online Git lessons and exercises [for credit].

Set aside some time each week to practice reading & writing Java code — and using Git.

